HACKTHEBOX

ANDROID PENETRATION TESTING AUTOMATION

CHEAT SHEET

HTB ACADEMY CHEATSHEET

Static Analysis Tools

Command	Description
`docker pull opensecurity/mobile-security-framework-mobsf:latest` `docker run -it --rm -p 8000:8000 opensecurity/mobile-security-framework-mobsf:latest`	Set up MobSF framework locally.

`python3 -m venv quark`
`source quark/bin/activate`
`pip3 install -U quark-engine`
`quark --version`
	Set up Quark Engine.

`quark -s -a myapp.apk -r quark-rules`	Provide us with an overview of the findings.
`quark -s -a myapp.apk -r quark-rules -t 100`	Only displays rules that have a confidence level of 100%.
`quark -s -a myapp.apk -r quark-rules --permission`	List all the permissions the application requests from the operating system.
`quark -s -a myapp.apk -r quark-rules --list all > output.txt`	List the methods included in an application.
`quark -s -a myapp.apk -r quark-rules --list custom | grep "methodName"`	Searching for targeted method names.
`quark -s -a myapp.apk -r quark-rules -t 100 -l detailed`	Adjusts the amount of detail we see about the labels linked to detected behaviors in the output.
`quark -s -a myapp.apk -r quark-rules -t 100 -c`	Search for potential malicious activities using specified rules, focused on crime-related behaviors.
`jadx-gui /full/path/myapp.apk`	Analyze the app using JADX
`quark -s -a myapp.apk -r quark-rules -t 100 -w report`	Create a web report.

Drozer

Command	Description
`openvpn ./corellium.com\ VPN\ -\ Default\ Project.ovpn`	Connect to the Corellium's network.
`adb connect 10.11.1.1:5001`	Connect to the remote device.
`adb install myapp.apk`	Install apps on the device.
`pipx install drozer`	Install Drozer.
`adb install drozer-agent.apk`	Install Drozer agent to the device.
`adb forward tcp:31415 tcp:31415` `drozer console connect 10.11.1.1:5001`	Forward port `31415` to connect to the server and gain interactive command execution via the `drozer` tool installed on the host machine.
dz> `list`	List available modules within Drozer.
dz> `run app.package.list -f MyApplication`	Run the module `run app.package.list` to list all packages installed on the device, and filter the results by the app name `MyApplication`.
dz> `run app.package.info -a com.hackthebox.myapp`	Gather some general information about this app
dz> `run app.package.manifest com.hackthebox.myapp`	Read the manifest file of the application.
dz> `run app.package.attacksurface com.hackthebox.myapp`	Indicates the application's components that are accessible to other apps or have the potential to be exploited.
dz> `run app.provider.info -a com.hackthebox.myapp`	Gather information about any Provider used by the app.
dz> `run scanner.provider.traversal -a com.hackthebox.myapp`	Scans the app for providers with Path Traversal vulnerability.
dz> `run app.provider.read content://com.hackthebox.myapp.fileprovider/../../../system/etc/hosts`	Exploit the vulnerable content provider and read the file `/system/etc/hosts`.
dz> `run scanner.provider.finduris -a com.hackthebox.myapp`	Gather more information about the provider's URIs.
dz> `run scanner.provider.injection -a com.hackthebox.myapp`	Scan the app for providers vulnerable to SQL injection.
dz> `run scanner.provider.sqltables -a com.hackthebox.myapp`	Exploits any SQL injection vulnerabilities found in the app.
dz> `run app.provider.query content://com.hackthebox.myapp.mycontentprovider/users/`	Query the default table associated with the the URI used by the provider `content://com.hackthebox.myapp.mycontentprovider/users/`.
dz> `run app.provider.query content://com.hackthebox.myapp.mycontentprovider/users/ --projection "_id, name"`	Exploit the `projection` method to read the contents of the `Users` and `Tokens` tables.
dz> `run app.provider.query content://com.hackthebox.myapp.mycontentprovider/users/ --projection "_id, name, (SELECT token FROM Tokens)"`	Fetch the content of this table `Users`.

Command	Description
dz> `run app.provider.insert content://com.hackthebox.myapp.mycontentprovider/users/ --int _id 2 --string name test`	Insert new entries into the database.
dz> `run app.provider.query content://com.hackthebox.myapp.mycontentprovider/users/ --projection "_id, name"`	List the contents of the `Users` table.
dz> `run app.provider.update content://com.hackthebox.myapp.mycontentprovider/users/ --selection "_id=?" --selection-args 2 --string name doe`	Update an existing entry in the database by exploiting the vulnerable URI.
dz> `run app.provider.delete content://com.hackthebox.myapp.mycontentprovider/users/ --selection "_id=?" --selection-args 2`	Delete existing entries.

Objection

Command	Description
`pip3 install objection`	Install Objection.

`wget https://github.com/frida/frida/releases/download/16.4.10/frida-server-16.4.10-android-arm64.xz`
`adb root`
`unxz frida-server-16.4.10-android-arm64.xz`
`mv frida-server-16.4.10-android-arm64 frida-server`
`adb push frida-server /data/local/tmp/`
`adb shell chmod +x /data/local/tmp/frida-server`
`adb shell /data/local/tmp/frida-server &`
	Set up Frida server on the device.

`pip3 install frida-tools` `pip3 install frida==16.1.11`	Install Frida tools locally.
`frida-ps -Uai`	List all apps and processes on a device with Frida.
`objection --gadget com.example.myapp explore`	Launch an exploration session for the app `com.example.myapp`, providing an interactive shell.
[usb] # `android hooking search classes com.example.supermarket`	Search for classes in a specific app.
[usb] # `android hooking search methods com.example.supermarket MainActivity`	Search for methods in the MainActivity class of an app.
[usb] # `android hooking list activities`	List all activities in an app.
[usb] # `android hooking list class_methods com.example.supermarket.MainActivity`	List methods of the MainActivity class in an app.
[usb] # `android hooking list services`	List all services in an app.
[usb] # `android hooking list receivers`	List all receivers in an app.
[usb] # `android hooking list classes`	List all classes in an app.

Command	Description
[usb] # `android root disable`	Disable root access on an Android device
[usb] # `android hooking set return_value com.hackthebox.myapp.MainActivity.isDeviceRooted false`	Set the return value of the `isDeviceRooted` method in the `MainActivity` class to `false`.
[usb] # `android sslpinning disable`	Disable SSL pinning on an Android device.
[usb] # `import snippet_test.js`	Import the JavaScript file `snippet_test.js` into the current project.

Medusa

Command	Description

`git clone https://github.com/Ch0pin/medusa.git`
`cd medusa`
`pip3 install -r requirements.txt`
`python3 medusa.py`
	Install Medusa locally.

medusa➤ `search bypass`	Search for a specific module or category.
medusa➤ `list -3`	List 3rd party apps installed on the device.
medusa➤ `list com.hackthebox.myapp path`	Retrieve specific paths related to the app.
medusa➤ `use root_detection/rootbeer_detection_bypass_no_obfuscation`	Select a specific module.
medusa➤ `compile`	Prepare the selected module.
medusa➤ `show mods`	List the selected modules.
medusa➤ `rem root_detection/rootbeer_detection_bypass_no_obfuscation`	Remove a specific module.
medusa➤`pad`	View or edit the contents of the scratchpad.
medusa➤ `run -f com.hackthebox.myapp`	Restart the app using Frida by attaching the compiled script.
medusa➤ `reset`	Remove all modules if more than one has been added.
medusa➤ `c cat modules/http_communications/system_http_proxy_get.med`	Execute shell commands outside Medusa.
medusa➤ `libs -j com.hackthebox.myapp --attach`	Search the app for native libraries.
medusa➤ `enumerate com.hackthebox.myapp libmyapp.so --attach`	Hook any functions included in a native library.

Command	Description
medusa➤ `hook -n` medusa➤ `compile` medusa➤ `run -n 0`	Hook the return value of the native function `getToken` by specifying it takes `0` arguments.

HTB ACADEMY CHEATSHEET
HTB ACADEMY CHEATSHEET